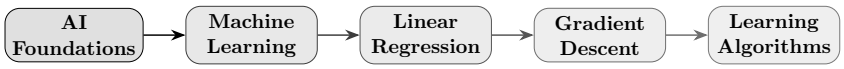
ACADEMIC REPORT

Artificial Intelligence & Machine Learning

From Foundations to Linear Regression & Learning Algorithms

Topics Covered:	Introduction to AI · History & Controversies Chatbots & Turing Test · Symbolic AI Linear Regression · Gradient Descent Learning Algorithms · Cost Functions
Level:	Undergraduate / Graduate
Year:	2026



Department of Computer Science

Prepared for Academic Study

Contents

1 Introduction to Artificial Intelligence **2**
1.1 What is Artificial Intelligence? . . . 2
1.2 Contemporary Applications of AI . . . 2
1.3 Strong AI vs. Weak AI . . . 3
1.4 History of AI . . . 4
1.5 AI Controversies . . . 4

2 Classical AI: Chatbots and the Turing Test **5**
2.1 The Turing Test . . . 5
2.1.1 Objections to the Turing Test . . . 5
2.2 Classical Chatbots . . . 6
2.2.1 ELIZA (1966) . . . 6
2.2.2 ALICE (Artificial Linguistic Internet Computer Entity) . . . 6
2.2.3 Kuki (formerly Mitsuku) . . . 7

3 Symbolic AI and the Physical Symbol System Hypothesis **8**
3.1 Symbolic AI . . . 8
3.2 Physical Symbol System Hypothesis (PSSH) . . . 9
3.2.1 What is a Physical Symbol System? . . . 9
3.2.2 Arguments For and Against PSSH . . . 9

4 Introduction to Machine Learning **10**
4.1 Datasets, Variables, and Features . . . 10
4.1.1 Types of Variables . . . 10
4.1.2 Key Terminology . . . 10
4.2 Categories of Learning Algorithms . . . 11
4.3 Supervised Learning . . . 11
4.3.1 Regression vs. Classification . . . 11
4.3.2 Common Supervised Learning Algorithms . . . 11
4.4 Unsupervised Learning . . . 12
4.4.1 Common Unsupervised Algorithms . . . 12
4.5 Semi-Supervised Learning . . . 12
4.5.1 Approaches to Semi-Supervised Learning . . . 13

5 Linear Regression **14**
5.1 Introduction to Linear Regression . . . 14
5.2 Multiple Linear Regression . . . 15
5.2.1 Weights, Bias, and Parameters . . . 15

5.3 Loss Function and Cost Function . . . 16
5.3.1 Mean Squared Error (MSE) . . . 16
5.4 Explained and Unexplained Variation . . . 16
5.4.1 Sum of Squares . . . 17
5.5 Coefficient of Determination (R^2) . . . 17
5.6 Correlation Coefficient . . . 18

6 Gradient Descent and Parameter Estimation 19
6.1 The Optimization Problem . . . 19
6.1.1 Types of Optimization Problems . . . 19
6.2 Gradient Descent . . . 19
6.2.1 Intuition . . . 19
6.2.2 Gradient Descent Update Rule . . . 20
6.2.3 Gradients for Simple Linear Regression (MSE) . . . 20
6.3 Gradient Ascent vs. Gradient Descent . . . 21
6.4 Learning Rate: Effect on Convergence . . . 21
6.5 Worked Example: Gradient Descent Iterations . . . 22
6.6 Variants of Gradient Descent . . . 23
6.7 Parameter Estimation: Analytical vs. Iterative . . . 23
6.7.1 Normal Equation (Analytical) . . . 23
6.7.2 Gradient Descent (Iterative) . . . 24

7 Summary and Key Formulas 25
7.1 Quick Reference: Key Formulas . . . 25
7.2 Learning Algorithm Comparison . . . 26

Chapter 1

Introduction to Artificial Intelligence

1.1 What is Artificial Intelligence?

Artificial Intelligence (AI)

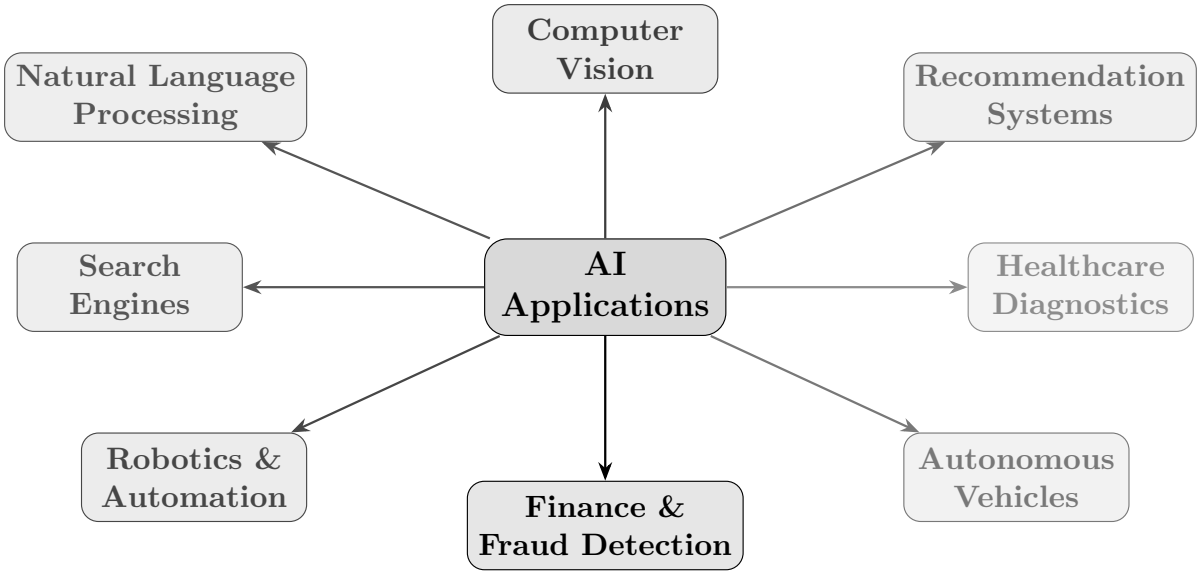
Artificial Intelligence is the branch of computer science concerned with building systems that can perform tasks that would normally require human intelligence. These tasks include reasoning, learning, planning, perception, language understanding, and problem solving.

AI can be broadly understood through two lenses:

- **Acting humanly:** Systems that behave as a human would (Turing test approach)
- **Acting rationally:** Systems that make optimal decisions given available information
- **Thinking humanly:** Cognitive modelling approach mimicking human thought
- **Thinking rationally:** Laws-of-thought approach using logic

1.2 Contemporary Applications of AI

AI is no longer a theoretical discipline — it is embedded in everyday life:



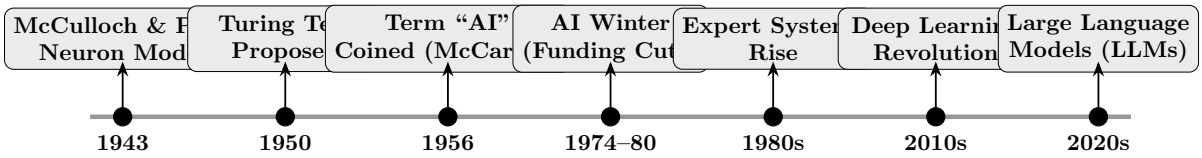
Domain	AI Application	Example
Healthcare	Disease detection	Cancer screening via imaging
Finance	Fraud detection	Credit card anomaly detection
Transport	Autonomous driving	Tesla Autopilot, Waymo
Entertainment	Recommendation	Netflix, Spotify
Language	Translation & chatbots	Google Translate, ChatGPT
Manufacturing	Quality control	Defect detection via vision

1.3 Strong AI vs. Weak AI

Weak AI (Narrow AI)	**Strong AI (General AI)**
Designed for a specific task	Capable of any intellectual task a human can do
Operates within predefined domain	Self-aware, conscious, general reasoning
Examples: Siri, chess engines, spam filters	Hypothetical — not yet achieved
Currently prevalent and practical	Remains a long-term research goal

Key Insight: All AI systems today are *Weak AI*. Strong AI (Artificial General Intelligence, AGI) remains a theoretical goal. The distinction matters for setting realistic expectations about what AI can and cannot do.

1.4 History of AI



1.5 AI Controversies

- **Bias and Fairness:** AI models trained on biased data perpetuate discrimination (e.g., facial recognition performing poorly on darker skin tones).
- **Privacy:** Mass surveillance and data collection by AI systems raise civil liberties concerns.
- **Job Displacement:** Automation threatens certain categories of employment.
- **Autonomous Weapons:** Lethal autonomous weapons systems (LAWS) raise profound ethical questions.
- **Deepfakes:** AI-generated media can be used to spread disinformation.
- **Black-box Problem:** Many modern AI systems (deep neural networks) are not interpretable.

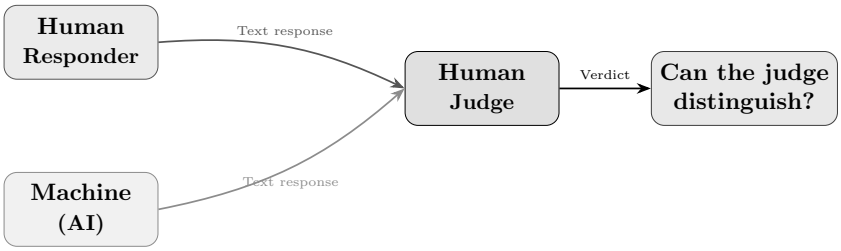
Chapter 2

Classical AI: Chatbots and the Turing Test

2.1 The Turing Test

Turing Test (1950)

Proposed by Alan Turing in his paper *"Computing Machinery and Intelligence"*, the Turing Test evaluates whether a machine can exhibit intelligent behaviour indistinguishable from that of a human. A human judge converses with both a human and a machine via text. If the judge cannot reliably distinguish which is which, the machine is said to have passed the test.



2.1.1 Objections to the Turing Test

1. **Chinese Room Argument (Searle, 1980):** A system can appear to understand language by manipulating symbols according to rules without actually *understanding* anything. Simulation $\neq$ intelligence.

2. **It measures imitation, not intelligence:** Passing requires deception skills, not genuine cognition.

3. **Human cognition as the wrong benchmark:** Intelligence need not resemble human intelligence.

4. **No standard judge:** Results vary with different judges and contexts.

5. **Narrow scope:** Only tests linguistic behaviour, ignoring vision, motor skills, and emotional intelligence.

2.2 Classical Chatbots

2.2.1 ELIZA (1966)

ELIZA

Developed by Joseph Weizenbaum at MIT, ELIZA was one of the first natural language processing programs. Its most famous script, **DOCTOR**, simulated a Rogerian psychotherapist by applying simple pattern matching and scripted responses. It had no understanding of the conversation — it simply reflected statements back as questions.

Example: ELIZA Interaction

User: "I feel that nobody cares about me."
ELIZA: "Why do you feel that nobody cares about you?"

User: "My mother never listened to me."
ELIZA: "Tell me more about your family."

Key mechanism: ELIZA looks for keywords like "mother," "feel," "nobody" and applies transformation templates to the user's input.

2.2.2 ALICE (Artificial Linguistic Internet Computer Entity)

ALICE

Created by Richard Wallace in 1995, ALICE uses **AIML (Artificial Intelligence Markup Language)** — an XML-based language for defining conversation rules. ALICE won the Loebner Prize (a Turing Test competition) three times (2000, 2001, 2004). It is a far more sophisticated pattern-matching system than ELIZA but still lacks genuine understanding.

Example: ALICE/AIML Rule Structure

```
<category>
<pattern>WHAT IS YOUR NAME</pattern>
<template>My name is ALICE.</template>
</category>
```

The pattern matches the user query; the template is the fixed response.

2.2.3 Kuki (formerly Mitsuku)

Kuki

Kuki, originally named Mitsuku, was created by Steve Worswick using AIML. It is one of the most advanced retrieval-based chatbots, having won the Loebner Prize five times. It maintains a consistent persona, handles multi-turn conversations better than ELIZA or ALICE, and can perform simple reasoning tasks. It is freely available at `kuki.ai`.

Chatbot	Year	Mechanism	Notable Feature
ELIZA	1966	Pattern matching (scripts)	Rogerian therapist simulation
ALICE	1995	AIML rule-based	Loebner Prize winner (3x)
Kuki	2005	Advanced AIML	Loebner Prize winner (5x)

Chapter 3

Symbolic AI and the Physical Symbol System Hypothesis

3.1 Symbolic AI

Symbolic AI

Symbolic AI (also called Good Old-Fashioned AI, or **GOFAI**) is an approach to AI based on the explicit representation of knowledge as symbols and the manipulation of those symbols according to formal rules. It includes expert systems, logic-based reasoning, and rule-based systems.

Key characteristics:

- Knowledge is **explicitly encoded** as facts, rules, or ontologies
- Reasoning is performed through **logical inference**
- The system is **interpretable** — you can trace every decision
- Struggles with **ambiguity, uncertainty, and learning from data**

Example: Expert System: Medical Diagnosis

IF patient has fever > 38.5°C
AND patient has cough
AND patient has fatigue
THEN possible diagnosis = Influenza
Such rule-based systems (like MYCIN in the 1970s) could diagnose diseases at expert level but could not learn from new cases without manual rule updates.

3.2 Physical Symbol System Hypothesis (PSSH)

Physical Symbol System Hypothesis (Newell & Simon, 1976)

"*A physical symbol system has the necessary and sufficient means for general intelligent action.*"

This hypothesis asserts that:

1. Intelligence can be reduced to symbol manipulation.
2. Any system capable of general intelligent action must be a physical symbol system.
3. The brain is itself a physical symbol system.

3.2.1 What is a Physical Symbol System?

A physical symbol system consists of:

- **Symbols:** Physical patterns (bits, words, tokens) that can be combined
- **Expressions:** Structures built from symbols
- **Processes:** Operations that create, modify, and destroy expressions

Example: Physical Symbol System: Arithmetic

Consider: $3 + 5 = 8$

Symbols: "3", "+", "5", "=", "8"
Expression: $3 + 5$
Process: Addition rule transforms $3 + 5 \to 8$

3.2.2 Arguments For and Against PSSH

For PSSH	Against PSSH
Successful expert systems in narrow domains	Fails to handle common-sense reasoning
Formal logic proved powerful for structured problems	Symbols have no inherent meaning (Searle)
Turing machines are symbol systems that are universal	Human cognition may not be symbolic at all
Language appears symbolic in structure	Frame problem: cannot handle dynamic real world

Chapter 4

Introduction to Machine Learning

4.1 Datasets, Variables, and Features

Dataset

A **dataset** is a structured collection of data points (observations or samples), where each data point is described by a set of **features** (also called variables or attributes) and often an associated **label** or **target value**.

Example: House Price Dataset

Size (m^2)	Bedrooms	Age (years)	Location	Price ($)
85	3	10	Urban	250,000
120	4	5	Suburban	320,000
60	2	25	Rural	150,000
200	5	2	Urban	600,000

Features (inputs): Size, Bedrooms, Age, Location — **Target (output):** Price

4.1.1 Types of Variables

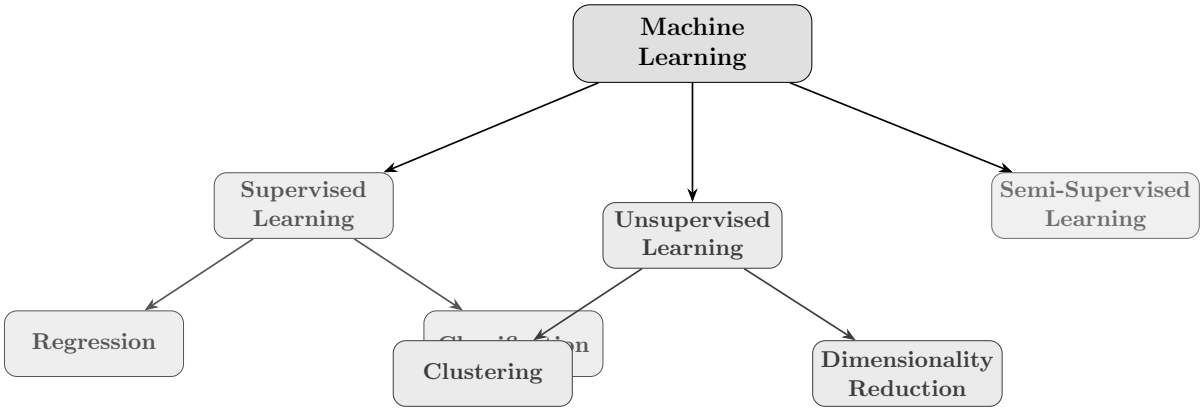
Type	Description	Example
Continuous	Any real number value	Height, Temperature, Price
Discrete	Integer or countable value	Number of bedrooms
Categorical (Nominal)	No natural ordering	Color (Red, Blue, Green)
Categorical (Ordinal)	Has natural ordering	Ratings (Low, Medium, High)
Binary	Only two values	Spam (Yes/No), Gender

4.1.2 Key Terminology

- **Feature** (x): An input variable used to make predictions
- **Target / Label** (y): The output we want to predict
- **Sample / Observation** ($x^{(i)}, y^{(i)}$): A single data point

- **Training set:** Data used to fit the model
- **Test set:** Data used to evaluate model performance
- **Feature vector:** $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$

4.2 Categories of Learning Algorithms



4.3 Supervised Learning

Supervised Learning

In supervised learning, the algorithm is trained on a **labelled dataset** — each input $\mathbf{x}^{(i)}$ has a corresponding known output $y^{(i)}$. The goal is to learn a mapping $f : \mathbf{x} \to y$ that generalises to unseen data.

4.3.1 Regression vs. Classification

	Regression	**Classification**
Output	Continuous value	Discrete class label
Example problem	Predict house price	Classify email as spam/not spam
Algorithms	Linear Regression, SVR, RF	Logistic Regression, SVM, KNN
Error metric	MSE, RMSE, MAE	Accuracy, F1-score, AUC

4.3.2 Common Supervised Learning Algorithms

- **Linear Regression:** Predicts a continuous output by fitting a linear relationship
- **Logistic Regression:** Predicts class probabilities using a sigmoid function
- **Decision Trees:** Partition the feature space using a tree structure
- **Random Forests:** Ensemble of decision trees to reduce overfitting
- **Support Vector Machines (SVM):** Finds the optimal hyperplane separating classes

- **K-Nearest Neighbours (KNN):** Classifies based on majority class of k closest training points
- **Neural Networks:** Multi-layer non-linear transformations for complex patterns

4.4 Unsupervised Learning

Unsupervised Learning

In unsupervised learning, the algorithm is given **unlabelled data**. The goal is to discover hidden structure, patterns, or groupings within the data without explicit guidance.

4.4.1 Common Unsupervised Algorithms

- **K-Means Clustering:** Partitions data into k clusters by minimising within-cluster variance

  Example: K-Means Example

  Given customer purchase data (no labels), K-Means might discover natural customer segments: "budget shoppers," "premium buyers," and "occasional buyers" — even though these labels were never provided.

- **DBSCAN:** Density-based clustering that can detect arbitrary-shaped clusters and outliers
- **Principal Component Analysis (PCA):** Reduces dimensionality by projecting onto axes of maximum variance
- **Autoencoders:** Neural networks that learn compressed representations of data
- **Gaussian Mixture Models (GMM):** Probabilistic model assuming data comes from multiple Gaussian distributions

4.5 Semi-Supervised Learning

Semi-Supervised Learning

Semi-supervised learning uses a **small amount of labelled data** combined with a **large amount of unlabelled data** for training. This is practical in real-world settings where labelling is expensive or time-consuming.

Example: Semi-Supervised in Medical Imaging

A hospital has 10,000 chest X-rays but only 500 have been reviewed by radiologists (labelled). Semi-supervised learning can use the pattern structure of all 10,000 images to improve a classifier trained only on the 500 labelled ones.

4.5.1 Approaches to Semi-Supervised Learning

- **Self-training:** Train on labelled data, predict on unlabelled, add high-confidence predictions as pseudo-labels
- **Co-training:** Train multiple models on different feature subsets, each labelling data for the other
- **Graph-based:** Propagate labels through a graph where similar points are connected
- **Generative models:** Use VAEs or GANs to model the data distribution

Learning Type	Labels	Data	Use Case
Supervised	All labelled	Any amount	Abundant labels available
Unsupervised	None	Unlabelled	No labels exist
Semi-supervised	Few labelled	Mostly unlabelled	Labelling is expensive

Chapter 5

Linear Regression

5.1 Introduction to Linear Regression

Linear Regression

Linear regression is a **supervised learning algorithm** that models the relationship between one or more input features and a continuous output variable by fitting a linear (straight-line) equation to the data.

The model assumes the output y is a linear function of the inputs:

Formula: Simple Linear Regression Model

$$\hat{y} = w_0 + w_1 x$$

where:

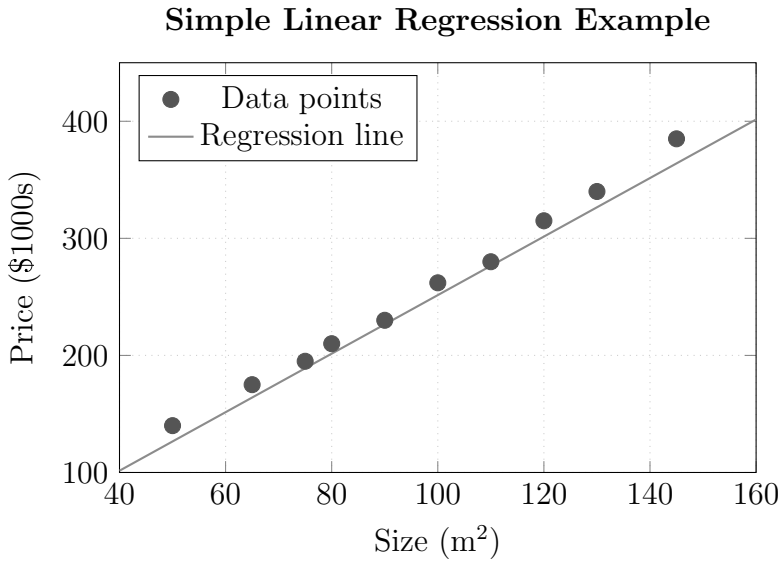
- $\hat{y}$ = predicted output (dependent variable)
- x = input feature (independent variable)
- w_1 = **weight** (slope) — how much y changes per unit change in x
- w_0 = **bias** (intercept) — value of $\hat{y}$ when $x = 0$

Example: House Price Prediction (Simple Linear Regression)

Predicting house price (y) from size in m^2 (x):

$$\hat{y} = 1500 + 2500 \cdot x$$

$x = 80$ m^2: $\hat{y} = 1500 + 2500 \times 80 = \$201{,}500$
$x = 120$ m^2: $\hat{y} = 1500 + 2500 \times 120 = \$301{,}500$
Interpretation: Each additional m^2 adds \$2,500 to the predicted price.



5.2 Multiple Linear Regression

Multiple Linear Regression

Multiple linear regression extends simple linear regression to **multiple input features**:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n = w_0 + \sum_{j=1}^{n} w_jx_j$$

In matrix form: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ (including bias as w_0 with $x_0 = 1$).

Example: Multiple Linear Regression: House Price

$$\hat{y} = 10{,}000 + 2500 \cdot x_{\text{size}} + 15{,}000 \cdot x_{\text{bedrooms}} - 500 \cdot x_{\text{age}}$$

$w_1 = 2500$: Each m^2 adds \$2,500
$w_2 = 15{,}000$: Each bedroom adds \$15,000
$w_3 = -500$: Each year of age reduces price by \$500
For a 100 m^2, 3-bedroom, 10-year-old house:

$$\hat{y} = 10{,}000 + 2500(100) + 15{,}000(3) - 500(10) = \$300{,}000$$

5.2.1 Weights, Bias, and Parameters

Term	Symbol	Role
Weight	w_j	Scales feature x_j; captures its importance
Bias	w_0 or b	Offset term; shifts the prediction up/down
Parameters	$\mathbf{w} = [w_0, w_1, \ldots, w_n]^T$	All learnable quantities in the model

The goal of training is to **estimate the best values** of weights $\mathbf{w}$ from the data.

5.3 Loss Function and Cost Function

Loss Function vs. Cost Function

- **Loss Function** $L(\hat{y}^{(i)}, y^{(i)})$: Measures the error on a **single** training example
- **Cost Function** $J(\mathbf{w})$: Measures the **average** loss over the **entire** training set

5.3.1 Mean Squared Error (MSE)

The most common loss for regression is the **Squared Error Loss**:

Formula: Squared Error Loss (single example)

$$L(\hat{y}^{(i)}, y^{(i)}) = \left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

Formula: Mean Squared Error (MSE) Cost Function

$$J(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right)^2 = \frac{1}{m}\sum_{i=1}^{m}\left(w_0 + w_1 x^{(i)} - y^{(i)}\right)^2$$

where m is the number of training examples.

Example: Computing MSE

i	$x^{(i)}$	$y^{(i)}$	$\hat{y}^{(i)} = 2 + 3x$	$(\hat{y}^{(i)} - y^{(i)})^2$
1	1	5	5	0
2	2	8	8	0
3	3	10	11	1
4	4	15	14	1

$$J(w_0 = 2, w_1 = 3) = \frac{1}{4}(0 + 0 + 1 + 1) = 0.5$$

5.4 Explained and Unexplained Variation

Decomposition of Total Variation

For any observation $y^{(i)}$ with mean $\bar{y}$:

$$\underbrace{(y^{(i)} - \bar{y})}_{\text{Total deviation}} = \underbrace{(\hat{y}^{(i)} - \bar{y})}_{\text{Explained}} + \underbrace{(y^{(i)} - \hat{y}^{(i)})}_{\text{Unexplained (residual)}}$$

5.4.1 Sum of Squares

Formula: Sum of Squares Decomposition

$$\underbrace{\sum_{i=1}^{m}(y^{(i)}-\bar{y})^2}_{SST\ \text{(Total)}} = \underbrace{\sum_{i=1}^{m}(\hat{y}^{(i)}-\bar{y})^2}_{SSR\ \text{(Regression/Explained)}} + \underbrace{\sum_{i=1}^{m}(y^{(i)}-\hat{y}^{(i)})^2}_{SSE\ \text{(Error/Unexplained)}}$$

$$SST = SSR + SSE$$

Quantity	Formula	Meaning
SST (Total Sum of Squares)	$\sum(y^{(i)}-\bar{y})^2$	Total variability in y
SSR (Regression Sum of Squares)	$\sum(\hat{y}^{(i)}-\bar{y})^2$	Variation explained by model
SSE (Error Sum of Squares)	$\sum(y^{(i)}-\hat{y}^{(i)})^2$	Variation not explained

5.5 Coefficient of Determination (R^2)

Formula: Coefficient of Determination

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

- $R^2 = 1$: Perfect fit — model explains all variation in y
- $R^2 = 0$: Model explains nothing; as good as predicting $\bar{y}$
- $R^2 < 0$: Model is worse than simply predicting the mean

Example: R^2 Calculation

$SST = 1000$ (total variability in house prices)
$SSE = 150$ (unexplained by our model)
$SSR = 850$ (explained by size, bedrooms, age)

$$R^2 = 1 - \frac{150}{1000} = 0.85$$

Interpretation: Our model explains 85% of the variability in house prices.

5.6 Correlation Coefficient

Formula: Pearson Correlation Coefficient

$$r = \frac{\sum_{i=1}^{m}(x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sqrt{\sum_{i=1}^{m}(x^{(i)} - \bar{x})^2 \cdot \sum_{i=1}^{m}(y^{(i)} - \bar{y})^2}}$$

- $r \in [-1, 1]$
- $r = 1$: Perfect positive linear correlation
- $r = -1$: Perfect negative linear correlation
- $r = 0$: No linear correlation
- **For simple linear regression:** $r^2 = R^2$

Chapter 6

Gradient Descent and Parameter Estimation

6.1 The Optimization Problem

Optimization Problem in ML

Training a machine learning model is fundamentally an **optimization problem**:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} J(\mathbf{w})$$

Find the parameter values $\mathbf{w}^*$ that **minimize** the cost function $J(\mathbf{w})$.

6.1.1 Types of Optimization Problems

- **Unconstrained:** Minimize $J(\mathbf{w})$ with no restrictions on $\mathbf{w}$ (most ML problems)
- **Constrained:** Minimize $J(\mathbf{w})$ subject to $g(\mathbf{w}) \leq 0$ or $h(\mathbf{w}) = 0$ (e.g., SVM)
- **Objective function:** The function being optimized (also called loss or cost in ML)

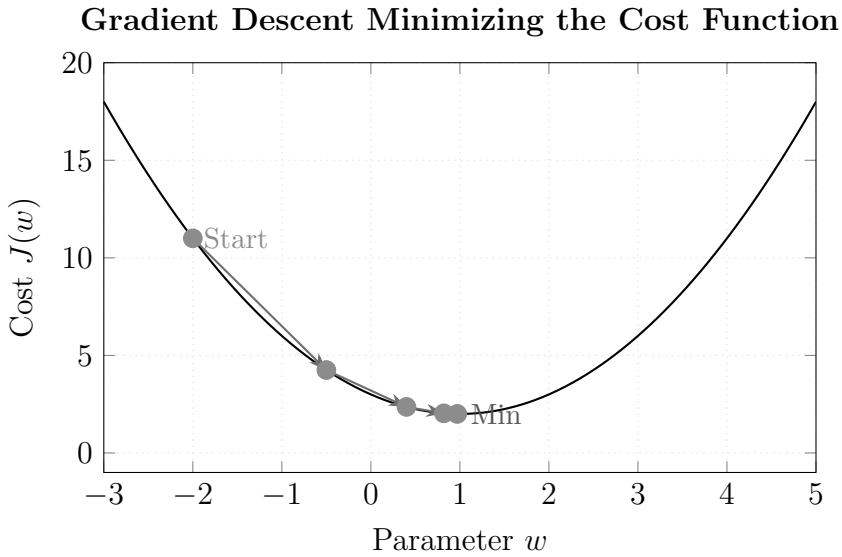
6.2 Gradient Descent

Gradient Descent

Gradient descent is an **iterative first-order optimization algorithm** used to minimize a differentiable function $J(\mathbf{w})$. Starting from an initial guess, it repeatedly moves in the direction of **steepest descent** (negative gradient) until convergence.

6.2.1 Intuition

Imagine standing on a hilly landscape (the cost function surface) in thick fog. You want to reach the lowest valley (minimum cost). Gradient descent says: at each step, feel the slope under your feet and take a step in the direction that goes **downhill most steeply**.



6.2.2 Gradient Descent Update Rule

Formula: Gradient Descent Update Rule

For each iteration t:

$$w_j \leftarrow w_j - \alpha \cdot \frac{\partial J(\mathbf{w})}{\partial w_j}$$

- α = **learning rate** (step size; hyperparameter)
- $\frac{\partial J}{\partial w_j}$ = partial derivative of cost w.r.t. weight w_j
- All weights are updated **simultaneously** in one pass

6.2.3 Gradients for Simple Linear Regression (MSE)

Given $J(w_0, w_1) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2$ and $\hat{y}^{(i)} = w_0 + w_1 x^{(i)}$:

Formula: Partial Derivatives for Simple Linear Regression

$$\frac{\partial J}{\partial w_0} = \frac{2}{m}\sum_{i=1}^{m}\left(w_0 + w_1 x^{(i)} - y^{(i)}\right)$$

$$\frac{\partial J}{\partial w_1} = \frac{2}{m}\sum_{i=1}^{m}\left(w_0 + w_1 x^{(i)} - y^{(i)}\right) \cdot x^{(i)}$$

The update rules become:

$$w_0 \leftarrow w_0 - \alpha \cdot \frac{2}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})$$

$$w_1 \leftarrow w_1 - \alpha \cdot \frac{2}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)}) \cdot x^{(i)}$$

6.3 Gradient Ascent vs. Gradient Descent

	Gradient Descent	**Gradient Ascent**
Goal	**Minimize** $J(\mathbf{w})$	**Maximize** $J(\mathbf{w})$
Update rule	$w \leftarrow w - \alpha \nabla J$	$w \leftarrow w + \alpha \nabla J$
Direction	Moves **against** gradient	Moves **with** gradient
Use case	Loss minimization, regression	Reward maximization (RL), likelihood

Key Insight: Any maximization problem can be converted to minimization by negating the objective: $\max J(w) \equiv \min -J(w)$. In machine learning, we almost always minimize a cost/loss function, so gradient descent is the standard.

6.4 Learning Rate: Effect on Convergence

Learning Rate α	**Behaviour**	**Problem**
Too large	Overshoots minimum	May diverge (cost increases)
Too small	Takes tiny steps	Very slow convergence
Just right	Steady convergence	Reaches minimum efficiently

6.5 Worked Example: Gradient Descent Iterations

Example: Step-by-Step Gradient Descent for Simple Linear Regression

Training data (4 points):
$(1, 2),\ (2, 4),\ (3, 6),\ (4, 8)$ (True relationship: $y = 2x$)
Initial parameters: $w_0 = 0,\ w_1 = 0$ **Learning rate:** $\alpha = 0.01$

Iteration 1:
Predictions: $\hat{y}^{(i)} = 0 + 0 \cdot x = 0$ for all i
Errors: $(0 - 2), (0 - 4), (0 - 6), (0 - 8)$

$$\frac{\partial J}{\partial w_0} = \frac{2}{4}\left[(-2) + (-4) + (-6) + (-8)\right] = \frac{2}{4}(-20) = -10$$

$$\frac{\partial J}{\partial w_1} = \frac{2}{4}\left[(-2)(1) + (-4)(2) + (-6)(3) + (-8)(4)\right] = \frac{2}{4}(-60) = -30$$

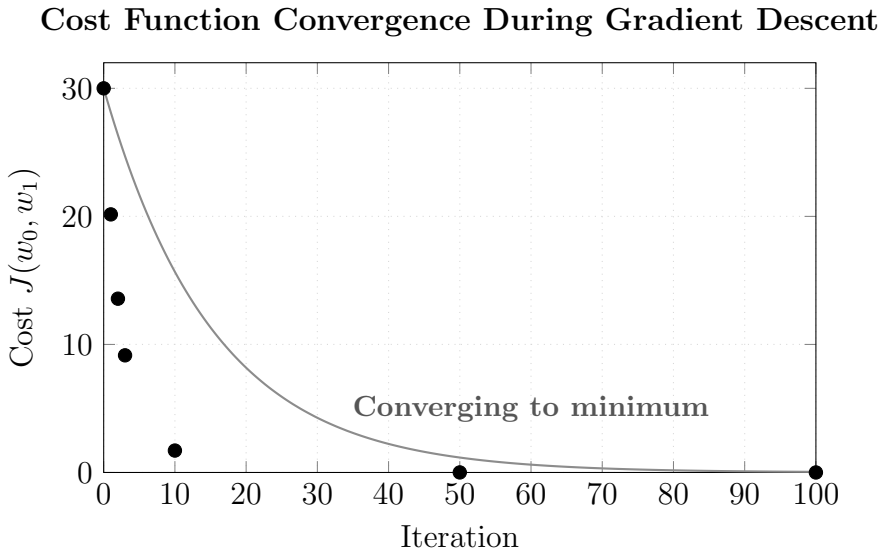
Updates:

$$w_0 \leftarrow 0 - 0.01 \times (-10) = 0.1$$

$$w_1 \leftarrow 0 - 0.01 \times (-30) = 0.3$$

Iteration	w_0	w_1	$J(w_0, w_1)$	$\partial J/\partial w_0$	$\partial J/\partial w_1$
0 (init)	0.000	0.000	30.000	-10.0	-30.0
1	0.100	0.300	20.155	-8.40	-25.4
2	0.184	0.554	13.572	-7.07	-21.4
3	0.255	0.768	9.148	-5.94	-18.0
10	0.531	1.527	1.712	-2.50	-7.6
50	0.982	1.967	0.003	-0.06	-0.2
100	0.999	1.999	≈ 0	≈ 0	≈ 0

After convergence: $w_0 \approx 0,\ w_1 \approx 2$ — matching the true relationship $y = 2x$!



6.6 Variants of Gradient Descent

Variant	Batch Size	Update frequency	Noise
Batch GD	All m samples	Once per epoch	None (smooth)
Stochastic GD (SGD)	1 sample	m times per epoch	High
Mini-Batch GD	k samples ($1 < k < m$)	m/k times per epoch	Moderate

Key Note

Mini-batch gradient descent is the **standard in practice**: batch sizes of 32, 64, or 128 balance computational efficiency and convergence stability. Libraries like PyTorch and TensorFlow use mini-batch GD by default.

6.7 Parameter Estimation: Analytical vs. Iterative

For linear regression there exist two approaches to finding $\mathbf{w}^*$:

6.7.1 Normal Equation (Analytical)

Formula: Normal Equation

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

- Directly computes the exact solution in one step
- Requires computing a matrix inverse: $O(n^3)$ complexity
- Impractical when n (number of features) is very large

6.7.2 Gradient Descent (Iterative)

Formula: Gradient Descent Parameter Estimation

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$

- Iteratively approaches the solution
- Scales to large datasets and many features
- Requires tuning the learning rate α
- Generalises to non-linear models (neural networks)

	Normal Equation	**Gradient Descent**
Speed	Fast for small n	Scales to large n
Hyperparameters	None	Needs α, iterations
Invertibility	Fails if $\mathbf{X}^T\mathbf{X}$ singular	Always applicable
Generalisation	Linear models only	Any differentiable model

Chapter 7

Summary and Key Formulas

7.1 Quick Reference: Key Formulas

Essential Formulas for Exam Review

Simple Linear Regression:

$$\hat{y} = w_0 + w_1 x$$

Multiple Linear Regression:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

MSE Cost Function:

$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})^2$$

Gradient Descent Update:

$$w_j \leftarrow w_j - \alpha \cdot \frac{\partial J}{\partial w_j}$$

Coefficient of Determination:

$$R^2 = 1 - \frac{SSE}{SST}, \quad SST = SSR + SSE$$

Pearson Correlation:

$$r = \frac{\sum (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sqrt{\sum (x^{(i)} - \bar{x})^2 \cdot \sum (y^{(i)} - \bar{y})^2}}$$

Normal Equation:

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

7.2 Learning Algorithm Comparison

Algorithm	Type	Output	Key Assumption
Linear Regression	Supervised	Continuous	Linear relationship
Logistic Regression	Supervised	Class label	Linear decision boundary
K-Nearest Neighbours	Supervised	Any	Proximity = similarity
Decision Tree	Supervised	Any	Greedy splits minimise impurity
K-Means	Unsupervised	Cluster ID	Spherical, equal clusters
PCA	Unsupervised	Components	Variance = information
Semi-supervised SVM	Semi-supervised	Class label	Unlabelled near boundary

End of Study Notes

These notes cover: Introduction to AI · Chatbots & Turing Test · Symbolic AI · Linear Regression · Gradient Descent · Learning Algorithms